

AASD 4010

Deep Learning - I

Applied AI Solutions Developer Program



PyTorch



Agenda



- PyTorch Basics
- Simple Neural Network
- Tensors
- Datasets & Dataloaders
- Transforms
- Model Building
- Autograds
- Optimizer
- Serialization



PyTorch Basics



PyTorch

Run PyTorch in a couple of ways:

- In the cloud: Use Colab with a GPU runtime to speed up operations Runtime > Change runtime type > GPU
- Locally: This option requires to setup PyTorch and TorchVision first on your local machine (<https://pytorch.org/get-started/locally/>)



PyTorch Installation (Windows)

PyTorch Build	Stable (1.8.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 10.2	CUDA 11.1	ROCm 4.0 (beta)	CPU
Run this Command:	conda install pytorch torchvision torchaudio cudatoolkit=10.2 -c pytorch			



PyTorch Installation (Mac)

PyTorch Build	Stable (1.8.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 10.2	CUDA 11.1	ROCm 4.0 (beta)	CPU
Run this Command:	<pre>conda install pytorch torchvision torchaudio -c pytorch # MacOS Binaries dont support CUDA, install from source if CUDA is needed</pre>			



Simple Neural Network



Simple Neural Network

- Import the necessary libraries
- Download Training and Test data from datasets
- Create Dataloaders
- Define the model
 - Linear layer
 - ReLU layer
- Define Loss function
- Define Optimizer
- Define Train function
- Define Test function
- Save the trained model
- Load the trained model
- Evaluate the model



PyTorch Tensors



Tensors

- Tensors are a **specialized data structure** that are very similar to arrays and matrices. In PyTorch, we use tensors to encode the inputs and outputs of a model, as well as the model's parameters
- Tensors are similar to NumPy's **ndarrays**, except that tensors can run on GPUs or other hardware accelerators
- Tensors and NumPy arrays can often share the same underlying memory, eliminating the need to copy data
- Tensors are also optimized for automatic differentiation



Tensors

- Initialize the Tensors
 - Directly from data
 - From a NumPy array
 - From a Tensor
 - From Random or Constant values
- Attributes of a Tensor
 - Shape
 - Datatype
 - Device



Tensors

- Availability of GPU
- Operations
 - Indexing
 - Slicing
 - Concatenating
 - Matrix Multiplication
 - Element-wise Multiplication
 - NumPy to Tensor
 - Tensor to NumPy



Datasets & Dataloaders



Datasets & Dataloaders

- PyTorch provides two data primitives:
 - `torch.utils.data.DataLoader`
 - `torch.utils.data.Dataset`
- Allows to use pre-loaded datasets as well as custom (own) data.
- Dataset stores the samples and their corresponding labels
- DataLoader wraps an iterable around the Dataset to enable easy access to the samples



Datasets

Load a dataset with the following parameters

- **root** is the path where the train/test data is stored,
- **train** specifies training or test dataset,
- **download=True** downloads the data from the internet if it's not available at root.
- **transform** and **target_transform** specify the feature and label transformations



Custom Dataset

A custom Dataset class must implement three functions:

- `__init__`
- `__len__`
- `__getitem__`

FashionMNIST images are stored in a directory **img_dir**, and their labels are stored separately in a CSV file **annotations_file**



Custom Dataset

The Dataset retrieves custom dataset's features and labels **one sample** at a time

While training a model, we typically want to pass samples in "minibatches", reshuffle the data at every epoch to reduce model overfitting, and use Python's multiprocessing to speed up data retrieval

DataLoader is an iterable that abstracts this complexity for us in an easy API



Dataloaders

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)

train_features, train_labels = next(iter(train_dataloader))
img = train_features[0].squeeze()
label = train_labels[0]
plt.imshow(img, cmap="gray")
plt.show()
```



Transforms



Transforms

Data does not always come in its final processed form that is required for training machine learning algorithms

Use transforms to perform some manipulation of the data and make it suitable for training

All torchvision datasets have two parameters

- **transform** to modify the features and
- **target_transform** to modify the labels, that accept callables containing the transformation logic

The **torchvision.transforms** module offers several commonly-used transforms out of the box



Building a Neural Network



Building a NN

- Import the necessary libraries
- Get Device for training
- Define the Class NeuralNetwork by
 - subclassing `nn.Module`, and
 - initialize the neural network layers in `__init__`
 - Every `nn.Module` subclass implements the operations on input data in the forward method
- Model Layers
 - `nn.Flatten`
 - `nn.Linear`
 - `nn.ReLU`
 - `nn.Sequential`
 - `nn.Softmax`



Autograds

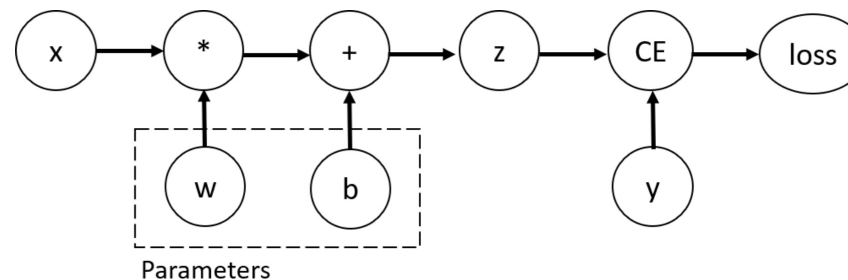


Autograds

- When training neural networks, the most frequently used algorithm is **back propagation**
- In this algorithm, **parameters** (model weights) are adjusted according to the **gradient** of the loss function with respect to the given parameter
- To compute those gradients, PyTorch has a built-in differentiation engine called **torch.autograd**
- It supports automatic computation of gradient for any computational graph
- Consider the simplest one-layer neural network:
 - input x ,
 - parameters w and b
 - some loss function
- `torch.randn(5, 3, requires_grad=True)`



Autograds



```
x = torch.ones(5) # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```



Optimizer



Optimizer

- Hyperparameters are adjustable parameters that let you control the model optimization process. Different hyperparameter values can impact model training and convergence rates (Hyperparameter Tuning)

https://pytorch.org/tutorials/beginner/hyperparameter_tuning_tutorial.html

- We define the following hyperparameters for training:
 - **Number of Epochs** - the number times to iterate over the dataset
 - **Batch Size** - the number of data samples seen by the model in each epoch
 - **Learning Rate** - how much to update models parameters at each batch/epoch
 - Smaller values yield slow learning speed, while large values may result in unpredictable behavior during training



Optimization Loop

- Once we set our hyperparameters, we can then train and optimize our model with an **optimization loop**
- Each iteration of the optimization loop is called an **epoch**.
- Each epoch consists of two main parts:
 - **The Train Loop** - iterate over the training dataset and try to converge to optimal parameters.
 - **The Validation/Test Loop** - iterate over the test dataset to check if model performance is improving.



Train Loop

```
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    for batch, (X, y) in enumerate(dataloader):
        # Compute prediction and loss
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if batch % 100 == 0:
        loss, current = loss.item(), batch * len(X)
        print(f"loss: {loss:>7f}    [{current:>5d}/{size:>5d}]"
```



Test Loop

```
def test_loop(dataloader, model, loss_fn):  
    size = len(dataloader.dataset)  
    test_loss, correct = 0, 0  
  
    with torch.no_grad():  
        for X, y in dataloader:  
            pred = model(X)  
            test_loss += loss_fn(pred, y).item()  
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()  
  
    test_loss /= size  
    correct /= size  
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```



Epoch

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Training Completed!")
```



Serialization



Saving & Loading Model parameters

Saving the model parameters

```
torch.save(model.state_dict(), 'model_weights.pth')
```

Loading the saved model parameters

```
model = models.vgg16()  
model.load_state_dict(torch.load('model_weights.pth'))  
model.eval()
```



Saving & Loading Model (Structure + Parameters)

Saving the model

```
torch.save(model, 'model.pth')
```

Loading the saved model

```
model = torch.load('model.pth')
```



Further Reading

PyTorch

<https://pytorch.org/tutorials/>

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial2/Introduction_to_PyTorch.html

PyTorch Cheat Sheet

<https://pytorch.org/tutorials/beginner/ptcheat.html>

